Fashion-MNIST Image Classification

Capstone experimental report | Linear baseline vs. a custom convolutional neural network

KEY RESULT		
	CNN test accuracy: 91.87%	**Macro-F1: 0.9182**
	Linear baseline: 83.86%	**Error-rate reduction vs. baseline: 49.6%**
	Gain: +8.01 percentage points	

Abstract

This project compares a multinomial linear classifier on flattened image pixels with a compact CNN trained from scratch on Fashion-MNIST. The same seeded train/validation split is used for both models, and the official 10,000-image test set is held out for the final evaluation. The CNN reaches 91.87% test accuracy and 0.9182 macro-F1, outperforming the linear baseline by 8.01 percentage points in accuracy. The remaining errors concentrate in visually similar apparel categories, especially Shirt, Coat, and neighbouring upper-body classes.

1. Problem and data

The task is supervised classification of 28 x 28 grayscale clothing images into 10 classes: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. Fashion-MNIST contains 60,000 official training images and 10,000 official test images. The supplied run uses a seeded split of 55,000 training and 5,000 validation examples from the official training partition; the official test set is not used for checkpoint selection or hyperparameter tuning.

Train	Validation	Test	Seed
55,000	**5,000**	**10,000**	**42**

2. Models and training choices

Linear baseline. Flatten 784 pixel values and map them directly to 10 class logits. It is an inexpensive reference point with no learned hierarchy of spatial features. **CNN.** Two convolutional blocks use 3 x 3 kernels, ReLU activations, and 2 x 2 max pooling; a 128-unit fully connected layer with dropout feeds the 10-class output. The CNN learns local patterns and combinations of edges and garment contours through weight sharing.



Convolutional feature extraction -> pooling -> classification head

Setting	Value	Reason
Loss	Cross-entropy on logits	Standard multi-class classification objective; logits are passed directly to the loss.
Optimizer	AdamW, lr 0.001	Adaptive updates with decoupled weight decay.
Batch / epochs	128 / max 12	Mini-batch optimization; early stopping uses validation loss (patience 3).
Regularization	Dropout 0.30, weight decay 0.0001	Reduce co-adaptation and discourage overly large weights.
Input scaling	Mean 0.2860, std 0.3530	Standardize pixel values consistently across splits.

Dataset source: Zalando Research Fashion-MNIST. Data and model weights are generated/downloaded at runtime and should not be committed as large files by default.

3. Evaluation protocol

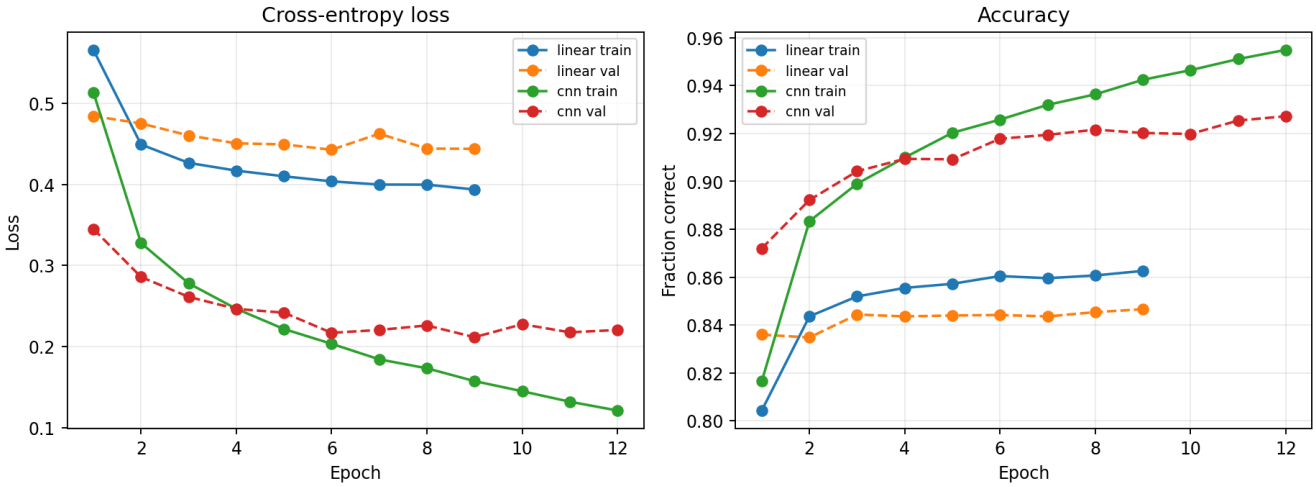
The primary metrics are test accuracy and macro-F1. Accuracy is the overall fraction of correctly classified test images. Macro-F1 is the unweighted mean of the per-class F1 scores, so each class contributes equally. Both models use the same data split and comparable optimizer settings. The selected checkpoint is the one with the lowest validation loss; the official test set is evaluated after selection.

Model	Test accuracy	Test macro-F1	Epochs run	Best epoch	Best val. loss	Train time
Linear baseline	0.8386	0.8393	9	6	0.4426	104.0 s
Custom CNN	0.9187	0.9182	12	9	0.2114	140.7 s

Main result

The CNN raises test accuracy from 83.86% to 91.87%, an absolute gain of **8.01 percentage points**. The error rate falls from 16.14% to 8.13%, a relative error reduction of **49.6%**. Macro-F1 increases from 0.8393 to 0.9182, suggesting that the improvement is not limited to aggregate accuracy alone.

Figure 1. Training and validation learning curves



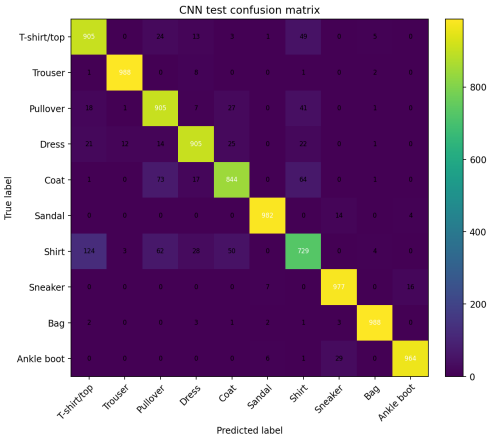
The CNN training loss decreases monotonically to 0.121, while its validation loss reaches a minimum of 0.211 at epoch 9, then ends at 0.220. Training accuracy finishes at 95.49% compared with validation accuracy of 92.72%. This train/validation gap and the post-epoch-9 validation-loss rise indicate some overfitting/plateauing; restoring the best validation-loss checkpoint is therefore appropriate. The CNN run completed 12 epochs, but the selected checkpoint was from epoch 9.

4. Class-level performance

The confusion matrix contains 10,000 test examples (1,000 per class). Class-wise recall below measures how many true examples of each category were recovered; precision measures how often predictions of that category were correct.

Class	Precision	Recall	F1	Correct / 1,000
T-shirt/top	0.844	0.905	0.874	905 / 1,000
Trouser	0.984	0.988	0.986	988 / 1,000
Pullover	0.840	0.905	0.871	905 / 1,000
Dress	0.923	0.905	0.914	905 / 1,000
Coat	0.888	0.844	0.866	844 / 1,000
Sandal	0.984	0.982	0.983	982 / 1,000
Shirt	0.803	0.729	0.764	729 / 1,000
Sneaker	0.955	0.977	0.966	977 / 1,000
Bag	0.986	0.988	0.987	988 / 1,000
Ankle boot	0.980	0.964	0.972	964 / 1,000

Figure 2. CNN confusion matrix



Rows = true class; columns = predicted class. Diagonal cells are correct classifications.

What the class metrics show

Strongest recalls: Trouser and Bag each reach 98.8% and 98.8% respectively; Sandal reaches 98.2% and Sneaker 97.7%. Their silhouettes or distinctive structure may make them easier to separate within this dataset.

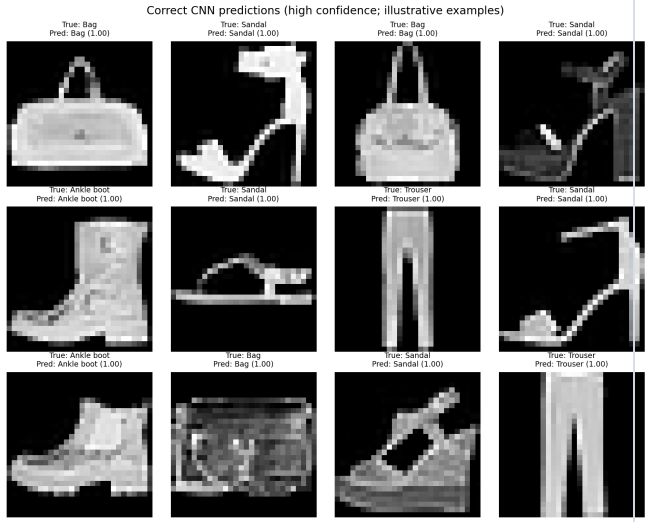
Weakest class: Shirt recall is 72.9% (729 of 1,000), with 124 Shirts misclassified as T-shirt/top and 62 as Pullover. Coat recall is 84.4%; notable Coat errors include 73 predicted as Pullover and 64 predicted as Shirt. These errors are consistent with visually similar upper-body garments at 28 x 28 resolution.

The per-class results make clear that the overall 91.87% accuracy hides uneven difficulty: the model performs very well on some categories but still confuses closely related clothing silhouettes.

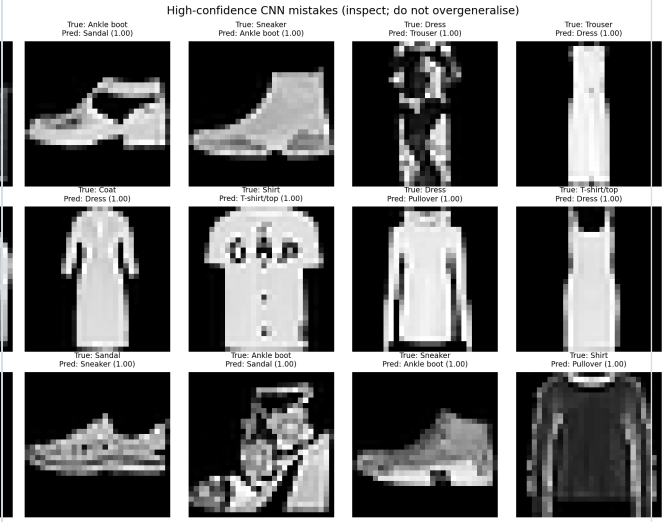
5. Error analysis: success and failure

The provided run artefacts contain both correct predictions and high-confidence mistakes from the held-out test set. These examples are illustrative cases, not a statistical estimate on their own.

Correct predictions



Misclassified examples



Concrete failure case: test index 23

True label	Predicted label	Predicted confidence
Ankle boot	**Sandal**	**99.9982%**

This is a concrete, high-confidence error: the true class is Ankle boot, but the model predicts Sandal with confidence approximately 99.9982%. The low resolution may hide cues such as the ankle shaft, while the visible lower outline may resemble a sandal in the compressed silhouette. This is a plausible explanation, not a verified causal diagnosis. The case is also a reminder that softmax confidence is not a guarantee of correctness; a calibration evaluation would be needed to quantify confidence reliability.

Other recurring confusions in the test matrix include Shirt -> T-shirt/top (124 examples), Shirt -> Pullover (62), Coat -> Pullover (73), and Coat -> Shirt (64). The matrix suggests the failures are not random: visually related upper-body classes account for many of them. Footwear errors also occur, including Ankle boot -> Sneaker (29 cases) and Sneaker -> Ankle boot (16).

A success example is test index 62, correctly classified as Bag with reported confidence 1.00. Such examples demonstrate that the model handles distinctive cases, but selected successes should not replace aggregate test metrics.

6. Conclusion, limitations, and next steps

The CNN is the stronger model for this experiment. On the fixed held-out Fashion-MNIST test set, it achieves 91.87% accuracy and 0.9182 macro-F1, compared with 83.86% and 0.8393 for the linear baseline. The 8.01-percentage-point accuracy gain supports the design choice to learn local spatial features rather than rely only on a linear mapping of flattened pixels. The result is meaningful for this benchmark, but not evidence of comparable performance on real-world photographs.

Limitations

First, Fashion-MNIST consists of small, centered grayscale images and is much less variable than unconstrained photographs. Second, only one seeded train/validation split and one reported run are available; run-to-run variance has not been estimated. Third, the CNN shows a gap between training and validation performance, and validation loss is lowest at epoch 9 despite training continuing to epoch 12. Fourth, the report does not record the exact GPU model or full software versions, so exact runtime replication cannot be guaranteed. Finally, a handful of chosen successes and failures are explanatory examples rather than a full human annotation study.

Next steps

A focused follow-up should (1) report per-class precision, recall, and F1 for each compared model; (2) repeat the experiment with several seeds and summarize mean and standard deviation; (3) use validation-only experiments to compare modest augmentation such as small translations; (4) inspect calibration with reliability plots or expected calibration error; and (5) test a small set of real-world images only as a qualitative robustness check, clearly separated from benchmark results. Any choices should be made using validation data rather than repeatedly consulting the official test set.

Reproducibility record

Item	Recorded value
Dataset split	55,000 train / 5,000 validation / 10,000 official test
Random seed	42 (Python, NumPy, PyTorch)
Optimizer / loss	AdamW, learning rate 0.001, weight decay 0.0001 / cross-entropy
Batch / epoch control	Batch size 128; maximum 12 epochs; patience 3 on validation loss
Runtime device	CUDA (specific GPU model was not saved in metrics.json)
Reported training time	Linear 104.04 s; CNN 140.71 s. Times depend on runtime hardware.
Reproduction command	`python -m src.train --model all --epochs 12 --batch-size 128 --seed 42` `python -m src.make_report --output report.pdf`

References

Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. https://github.com/zalandoresearch/fashion-mnist

PyTorch contributors. *torchvision.datasets.FashionMNIST API*. https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.FashionMNIST.html

Final note: metric values in this report are transcribed from the supplied run artefacts; class-level figures are computed from the included CNN confusion matrix. The PDF does not substitute generated values for missing experimental results.